

Team access — how everyone reads and edits the SSOT

The repo is the source of truth, but **most of the team will never open GitHub**. That's by design. Access comes in three tiers, mapped to what each person actually does. The golden rule that makes it all safe:

One-way out, never two-way. *The repo is the single source of truth. Other surfaces (Notion, Slack) read from it and propose changes to it — they never become a second place truth lives. The moment two surfaces both hold "the truth," you have no source of truth.*

The three tiers

Tier	Who	Surface	Reads?	Edits?	Needs a GitHub account?
1. Ask & edit	Everyone	The agent, in Slack	Yes — ask in plain language	Yes — say the change; agent opens a PR	No
2. Browse	Anyone	Published Notion views (read-only)	Yes — a few key pages	No (edits go via Tier 1)	No
3. Review & merge	Founder / maintainer (1–2 people)	GitHub	Yes — everything	Yes — approve/merge PRs	Yes

The mental model to give the team: **Notion is the noticeboard, Slack is the reception desk, GitHub is the vault**. You read the noticeboard, you talk to reception to change anything, and only the keyholder opens the vault.

Tier 1 — Ask & edit (the primary interface for everyone)

A teammate talks to the company agent in Slack:

- "What's our pricing for the Team tier?" → the agent answers from `sales/pricing.md` .
- "Acme's renewal moved to March." → the agent opens a PR appending the timeline entry and updating compiled truth; the owner merges.

This is **better** than letting people edit files directly, not a compromise: the agent enforces the contracts (routing, frontmatter, compiled-truth-vs-timeline, the hard prohibitions) that a human editing raw Markdown would quietly break — and every change still flows through the PR gate with full provenance (the commit carries a `Co-Authored-By` agent trailer; the PR body cites who asked). Nobody needs git, Markdown, or a GitHub account to contribute truth.

Wiring options are below (§ Wiring the Slack agent).

Tier 2 — Browse (one-way published views)

For the handful of pages people actually *browse* rather than ask about — e.g. `sales/pricing.md` , `ops.md` , `GLOSSARY.md` , the latest `weekly/` brief, `company.md` , `strategy.md` — publish read-only rendered copies into a Notion section. Rules that keep this safe:

- **Strictly one-way: repo → Notion, never back.** The Notion pages are *artifacts* (like a generated PDF), not a second truth. An automation re-renders them on every merge to `main`.
- **Stamp every published page** with a header: *"Generated from the company SSOT on <date> . commit <sha> — don't edit here. To change anything, ask the agent in #intelligence."*
- **Set Notion permissions to read-only** for the team on those pages. If someone edits anyway, the next publish overwrites it — which is correct, and the header warns them.
- **Publish a curated subset, not the whole repo.** Internal-only material (`people/` , `finance.md` , `decisions/` , `raw meetings/`) stays out of the published view — drive the selection off the audience: `frontmatter field ( team / public publish; internal never does)`.

A reference implementation (GitHub webhook on push to `main` → render changed files → Notion API, one-way) is a standard automation; for n8n users, see the agency toolkit's `publish-to-notion` building block. A GitHub Action does the same. Either way it is **publish**, not **sync**.

Tier 3 — Review & merge (GitHub, for the few)

Only the founder/maintainer (and a consultant during an engagement) needs a GitHub account — `CODEOWNERS` already encodes who must approve what. Their whole job is one page: [HUMAN-JOB.md](#) (incl. click-by-click PR review). Note: GitHub free private repos allow unlimited read-only collaborators, so a curious teammate *can* be invited to browse the raw repo — but Tiers 1–2 mean nobody has to.

Why not two-way sync? (the one thing never to build)

Two-way mirroring (repo ⇌ Notion, repo ⇌ a wiki) reintroduces every problem the SSOT exists to kill:

1. **Dual-write → no source of truth.** Someone edits in Notion while an agent edits the repo; a sync engine has to *guess* the winner. "There is only *the* version" dies first.
2. **It bypasses the gate.** A synced-back edit lands as truth with no PR, no diff, no review, no CI (the secret-scan and frontmatter checks never run), and no provenance. The entire audit trail — the thing that makes this trustworthy — is gone.
3. **Format/metadata corruption.** Every bidirectional sync engine eventually mangles frontmatter, ordering, or wikilinks, silently.

This is the same reason `guides/ARCHITECTURE.md` forbids mirroring live systems into the repo. Read from the SSOT freely; write to it only through the gate.

Wiring the Slack agent

The repo's headline promise — *"ask a question in Slack, get an answer grounded in the repo"* — needs an agent process connected to both Slack and this repo. Three common setups, simplest first:

Option	What it is	Best when
Claude in Slack	Anthropic's Claude app/agent in your Slack workspace, pointed at the repo (via the GitHub MCP server or a cloned working copy).	Fastest start; you already use Claude.
OpenClaw in a channel	An OpenClaw agent whose workspace <i>is</i> this repo (see ../agents-compatible/openclaw/README.md), bridged to a Slack channel.	You want a persistent, self-hosted agent with memory.

n8n (or similar) webhook bridge	Slack slash-command / Events API → n8n → Claude API with repo context + GitHub write-back as a PR.	You already run n8n and want full control of the loop.
--	--	--

Whichever you choose, hold these invariants:

- **Reads** can be broad (the agent can read any non-secret file).
- **Writes** are always PRs to a branch — never direct commits to `main` — carrying the `Co-Authored-By` agent trailer and citing the requester.
- **Scope the agent's GitHub token** to this one repo, least-privilege; rotate per `SECURITY.md`.
- **One workspace = one repo.** Never let a single agent write across multiple company SSOTs.

Set the channel (e.g. `#intelligence`) and put its name in the Tier-2 published-page header so browsers know where to go to request a change.

Quick decision guide

- *"I just want to look something up."* → Ask the agent in Slack (Tier 1), or read the Notion view (Tier 2).
- *"I need to change a fact."* → Tell the agent in Slack. It opens the PR. (Tier 1)
- *"I approve changes / I own the repo."* → GitHub (Tier 3); your job is [HUMAN-JOB.md](#).
- *"Can we let Notion edits flow back to the repo?"* → **No**. See § Why not two-way sync.